	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): Proyecto – Etapa 2

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

No. de lista o brigada: 6

Semestre: 3ro

Fecha de entrega: 26/10/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **3**
1.1. Planteamiento del problema 3
1.2. Motivación 3
1.3. Objetivos 3

2. Marco Teórico **4**

3. Desarrollo **6**
3.1. Diseño Estructural: Interfaces y Clases Abstractas 6
3.2. Encapsulamiento, Visibilidad y Construcción de Objetos 6
3.3. Especialización y Sobrescritura de Métodos (`@Override`) 6
3.4. Polimorfismo y Control Central (La Clase `Sistema`) 7
3.5. Entorno de Pruebas (La Clase `Main`) 7

4. Resultados **9**

5. Conclusiones **11**

1. Introducción

1.1. Planteamiento del problema

El proyecto actual es una continuación del simulador de sistema hospitalario desarrollado en la primera etapa. Para esta segunda etapa, el sistema evoluciona mediante la aplicación del encapsulamiento, paquetes, herencia y polimorfismo. Se incorporan clases abstractas e interfaces para representar de manera más fiel y estructurada los diferentes tipos de personal médico y pacientes.

El sistema conserva las funcionalidades operativas previas (restricciones de asignación por especialidad y límites de pacientes), pero transforma su arquitectura interna para soportar diferentes modalidades de atención (general y especialidad) utilizando estructuras de datos unificadas que aprovechan el polimorfismo.

1.2. Motivación

El objetivo de esta etapa es reestructurar lo obtenido en la etapa 1 para que sea más robusto, escalable y seguro. Al justificar la visibilidad de los atributos, organizar el código en paquetes lógicos y abstraer comportamientos comunes en clases base, se simula un entorno de desarrollo profesional y se resuelven problemas de duplicación de código detectados en la etapa inicial.

1.3. Objetivos

- **General:** Evolucionar la arquitectura del simulador hospitalario aplicando encapsulamiento, herencia, clases abstractas, interfaces y polimorfismo.
- **Específicos:**
  - Aplicar el encapsulamiento con modificadores de acceso (`private`, `protected`).
  - Organizar el código fuente mediante el uso de `package`, `import` y *Fully Qualified Names*.
  - Diseñar jerarquías de herencia estableciendo `Paciente`, `Medico` y `Enfermero` como clases abstractas con sus respectivas subclases (General, Especialista, Jefe de Enfermería).
  - Demostrar el polimorfismo estático y dinámico almacenando diferentes subclases en estructuras de datos unificadas basadas en sus clases padre.
  - Implementar la interfaz `AtencionMedica` y empaquetarla opcionalmente como archivo JAR.

2. Marco Teórico

1. **Abstracción:** La abstracción es el proceso de identificar las características (atributos) y acciones o comportamientos (métodos) propios de un elemento que deseamos representar. [1]

2. **Objeto:** Representan cosas del mundo real, los elementos de un objeto se dividen en dos categorías principales atributos y métodos. [9]

3. **Encapsulamiento:** Sucede cuando algo es envuelto en una capa protectora. Cuando el encapsulamiento se aplica a los objetos, significa que los datos del objeto están protegidos, "ocultos" dentro del objeto, el resto del programa no puede acceder de manera directa a los datos de un objeto lo tiene que hacer con ayuda de los métodos del objeto. [5]

4. **Modificadores de acceso:** Los modificadores de acceso (o de visibilidad) establecen permisos y nivel de accesibilidad de los miembros (atributos y métodos) de una clase hacia otras partes del programa, lo que permite proteger los datos y aplicar el principio de encapsulamiento. [8]

5. **Public (Público):** El miembro es accesible desde cualquier otra clase u objeto en todo el sistema. [8]

6. **Protected (Protegido):** El miembro es accesible para la propia clase, para clases en el mismo paquete y fundamentalmente para sus clases hijas (subclases) mediante la herencia. [8]

7. **Private (Privado):** El miembro solo es visible desde dentro de la propia clase que lo define. Oculta el dato o método al resto del programa. [8]

8. **Métodos de acceso:** Son el medio para acceder a los atributos privados de un objeto y manipularlos. Estos métodos, generalmente llamados getters (para leer) y setters (para modificar). [5]

9. **La clase Object:** Es la raíz de la jerarquía de clases.Toda clase tiene Object como superclase. Todos los objetos, incluidos los arreglos, implementan los métodos de esta clase. [4]

10. **Herencia:** Permite crear nuevos objetos que asumen las propiedades de objetos existentes permitiendo agregar más funcionalidades. [6]

11. **Súper clase:** Una clase que es usada como base para heredarse. [6]

12. **Subclase:** Una clase que hereda de una súper clase. La clase derivada hereda todas las propiedades y métodos visibles de la clase base y, además, puede agregar propiedades y métodos propios. [6]

13. **IS-A:** (es un) Se basa en la herencia y permite afirmar que un objeto es de un tipo (clase) en específico. [8]

14. **HAS-A:** (tiene un) Se basa en el uso más que en la herencia, es decir, una clase X HAS-A Y si el código en la clase X tiene como atributo una referencia de la clase Y. [8]

15. **Sobrescritura:** La sobrescritura se refiere a la habilidad de redefinir el comportamiento de un método específico en una subclase, generando así un comportamiento acorde a las necesidades de cada clase. [8]

16. **toString:** Es definido dentro de la clase Object. Este método es utilizado para mostrar información de un objeto, para mostrar la información deseada es necesario sobrescribir el método. Debe tener el mismo nombre. Debe tener el mismo tipo y número de parámetros. El tipo de nivel de acceso debe ser igual o más accesible. El valor de retorno debe ser del mismo tipo o un subtipo. [8]

17. **Super:** Dentro de una clase derivada se puede acceder a los elementos de la clase base a través de la palabra reservada super. [8]

18. **Polimorfismo:** Palabra de origen griego que significa "muchas formas". Se refiere a la propiedad por la que es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos. Consiste en conseguir que un objeto de una clase se comporte como un objeto de cualquiera de sus subclases. Se puede aplicar tanto a métodos como a tipos de datos. Los métodos pueden evaluar y ser aplicados a diferentes tipos de datos de manera indistinta. Los tipos polimórficos son tipos de datos que contienen al menos un elemento, cuyo tipo no está especificado. [7]

19. **Polimorfismo dinámico o paramétrico:** Es aquel en el que no se especifica el tipo de datos sobre el que se trabaja y, por ende, se puede recibir utilizar todo tipo de datos compatible. Este tipo de polimorfismo también se conoce como programación genérica. [7]

20. **Polimorfismo estático (o ad hoc):** Es aquel en el que los tipos de datos que se pueden utilizar deben ser especificados de manera explícita antes de ser utilizados. [7]

21. **instanceof:** Permite identificar el tipo de objeto enviado, es decir, no revisa la referencia sino el objeto en sí.[7]

22. **Clases abstractas:** Una clase abstracta es una clase de la que no se pueden crear objetos, debido a que define la existencia de métodos, pero no su implementación, sus métodos son explícitos e implícitos. [7]

23. **Interfaces:** Este tipo de diseños permiten establecer la forma de una clase (nombres de métodos, listas de argumentos y tipos de retorno), pero no bloques de código. Las interfaces pueden ser implementadas por cualquier clase. La clase que implementa una interfaz está obligada a definir (implementar) los métodos que la interfaz declaró. Todos los datos miembros (atributos) que se definen en una interfaz son públicos, estáticos y finales y dado que las interfaces no pueden instanciarse, resultan una buena herramienta para implantar grupos de constantes que pueden ser llamados sin crear objetos. [7]

3. Desarrollo

Para esta segunda etapa, el sistema evolucionó de un conjunto de clases simples a una arquitectura robusta basada en contratos (interfaces), jerarquías de herencia y protección rigurosa de datos.

3.1. Diseño Estructural: Interfaces y Clases Abstractas

El rediseño comenzó abstrayendo las acciones principales del hospital en tres interfaces: `AtencionMedica`, `AtencionEnfermero` y `AtencionPaciente`. En ellas **solo se declaran las firmas de los métodos**, sin incluir su lógica interna.

Ejemplo: Al crear la interfaz `AtencionMedica`, obligamos a que cualquier tipo de médico que se programe en el futuro (general, especialista, etc.) garantice la existencia de estos métodos. La interfaz no dicta *cómo* el médico debe dar consulta, solo exige que *pueda* hacerlo.

Estas interfaces fueron implementadas por las clases base de la Etapa 1 (`Medico`, `Enfermero` y `Paciente`), las cuales se transformaron en **clases abstractas**. Esto significa que el sistema ya no permite instanciar un objeto genérico (ej. `new Medico()`); forzosamente se debe especificar el tipo de médico, estableciendo una base sólida para la herencia.

3.2. Encapsulamiento, Visibilidad y Construcción de Objetos

Un cambio fundamental respecto a la Etapa 1 fue la modificación de la visibilidad de los atributos en las clases base, pasando de `private` a `protected`.

Si un atributo como `nombre` se declara `private`, queda completamente oculto, incluso para las clases hijas que heredan de él (como `MedicoGeneral`). Esto obligaría a las subclases a utilizar métodos *getters* internamente solo para leer sus propios datos heredados, lo cual es ineficiente. Al usar `protected` permitimos que las subclases hereden y accedan de forma directa a estos atributos para realizar validaciones, manteniendo un encapsulamiento estricto hacia el exterior (las clases `Sistema` o `Main` siguen sin poder modificarlos directamente).

Para la construcción de los objetos, se diseñó un flujo de inicialización descendente apoyado en la palabra reservada `super`. **Ejemplo:** Cuando se crea un `MedicoEspecialista`, su constructor recibe todos los datos requeridos e invoca inmediatamente a `super(...)`. Esta instrucción viaja a la superclase `Medico`, inicializa los atributos protegidos (nombre, cédula, instanciación de la lista de pacientes) y regresa a la subclase para inicializar el atributo privado y exclusivo `subEspecialidad`.

No se crearon métodos modificadores (*setters*) para atributos como `cedula` o `especialidad`. El diseño dicta que, en el contexto hospitalario, la cédula profesional de un médico es inmutable; nace con el objeto y no tiene justificación operativa para alterarse en tiempo de ejecución. Solo se permitieron setters en variables de estado dinámico (como las banderas booleanas `enConsulta` de los pacientes).

3.3. Especialización y Sobrescritura de Métodos (`@Override`)

La herencia no solo sirve para reutilizar código, sino para extenderlo. El requerimiento de la Etapa 2 establecía que las subclases debían incorporar al menos un atributo propio y

un comportamiento único. Las clases abstractas delegan los comportamientos que dependen del contexto a sus subclases mediante métodos abstractos. Al implementarlos, las subclases utilizan la anotación `@Override` para indicar que están sobrescribiendo la regla general con una regla específica.

Ejemplo de especialización lógica: En `MedicoGeneral`, el método `agregarPaciente` aplica la regla de que un médico general solo puede recibir pacientes generales evaluando con `instanceof`. Esta lógica es distinta en `MedicoEspecialista`, donde además de usar `instanceof` para buscar un `PacienteEspecialidad`, se evalúa mediante un `.equals()` que las cadenas de texto de ambas especialidades coincidan exactamente.

Ejemplo de expansión de responsabilidades: Mientras que `EnfermeroEspecialista` se limita a atender pacientes de su nicho, la clase `JefeEnfermeria` rompe el molde agregando una nueva responsabilidad: la gestión de personal. Se le añadieron los atributos `areaSupervision` y una lista interna `enfermerosACargo`. Su método exclusivo `supervisarEnfermeros()` itera sobre su propia lista de subordinados para imprimir sus métricas, demostrando que una subclase puede tener responsabilidades administrativas que su clase padre desconoce.

Sobrescritura de `toString()`: En todas las clases concretas se implementó la sobrescritura de este método heredado de la clase cósmica `Object`. Al utilizar `@Override`, se proporciona una representación en *String* del estado actual del objeto, permitiendo imprimir los atributos privados específicos (como `subEspecialidad`) en lugar de un código hash en memoria, facilitando el monitoreo y la depuración del sistema.

3.4. Polimorfismo y Control Central (La Clase `Sistema`)

La ventaja definitiva de esta arquitectura se evidencia en la clase `Sistema`, la cual actúa como el orquestador principal del hospital centralizando las estructuras de datos y la validación de la lógica de negocio.

En lugar de tener listas separadas para médicos generales y especialistas, la clase `Sistema` declara sus `ArrayList` basándose en las clases abstractas (`List<Medico> medicos`). Esto consolida la información gracias al **polimorfismo**.

Además, centraliza los métodos sobrecargados como `asignarPaciente(Paciente, Medico)`. Este método recibe argumentos genéricos (ignora temporalmente qué subclase específica está manejando) y actúa como un filtro. A través del polimorfismo y las condicionales `instanceof`, el sistema "desempaca"los objetos asegurando que las reglas del hospital se cumplan (límites de pacientes, coincidencia de especialidades) antes de invocar `medico.agregarPaciente(paciente)`. En ese momento ocurre el **despacho dinámico (*dynamic dispatch*)**: la Máquina Virtual de Java detecta el objeto real (ej. un `MedicoEspecialista`) e invoca automáticamente la versión del método sobrescrita en esa clase particular.

3.5. Entorno de Pruebas (La Clase `Main`)

El archivo `Main.java` es el punto de entrada donde se instancian los objetos y se simula el flujo de trabajo del hospital.

En el `Main` se evidencia el **Polimorfismo de Inclusión**. Declaramos una variable de tipo `Medico`, pero la instanciamos como un `MedicoEspecialista`. Cuando pasamos este objeto

al método `asignarPaciente` del `hospital`, el sistema interactúa con él sin saber a priori su subtipo exacto.

Posteriormente, al simular el ciclo clínico invocando `darConsulta()`, `finalizarConsulta()` y `darTratamiento()`, se comprueba la comunicación entre objetos: el médico modifica el estado interno del paciente (cambiando sus atributos booleanos `enConsulta`), lo que a su vez es validado por el enfermero antes de poder administrarle el tratamiento. El `Main` demuestra que el diseño encapsulado funciona, ya que los estados cambian fluidamente sin que la clase principal modifique un solo atributo directamente.

4. Resultados

Se demuestra el uso del polimorfismo mediante el almacenamiento de objetos de diferentes subclases (generales y especialistas) dentro de colecciones unificadas referenciadas por sus clases padre abstractas.

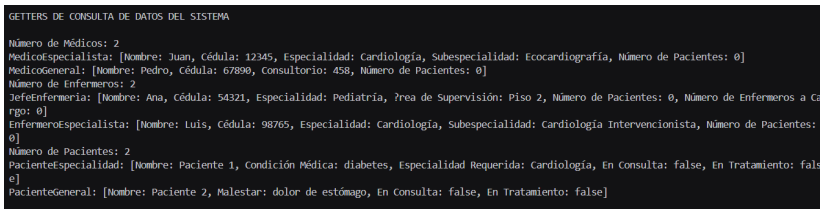


Figura 1: Colecciones unificadas aplicando polimorfismo dinámico.

Se evidencia la sobrescritura de métodos abstractos. Al iterar sobre la lista de pacientes y llamar al método `solicitarConsulta()`, el sistema ejecuta dinámicamente el comportamiento específico según el tipo real del objeto (Paciente General o Paciente de Especialidad).

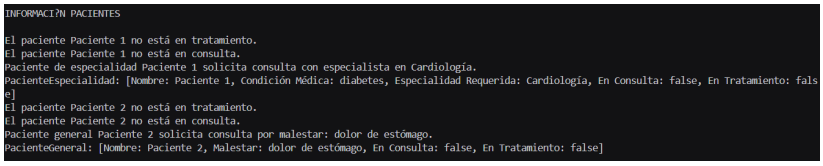


Figura 2: Sobrescritura de métodos y comportamiento dinámico.

Se comprueba la validación estricta de tipos cruzados en el método sobrecargado `asignarPaciente`. El sistema evalúa mediante `instanceof` las combinaciones, rechazando cruces inválidos (ej. paciente general con médico especialista) y aprobando coincidencias exactas.

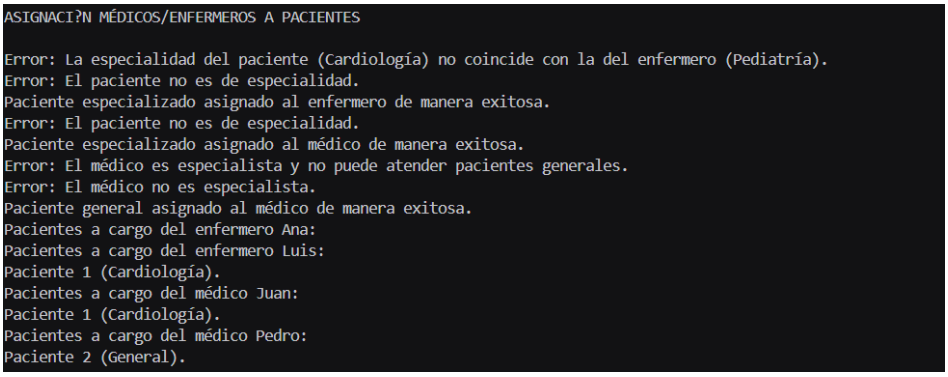


Figura 3: Validación de asignaciones mediante evaluación de tipos (`instanceof`).

Se verifica la correcta implementación del método `toString()` en cada subclase, permitiendo que la terminal imprima el estado interno y los atributos privados específicos (como consultorio o subespecialidad) de cada objeto instanciado sin necesidad de llamar a sus *getters* individualmente.

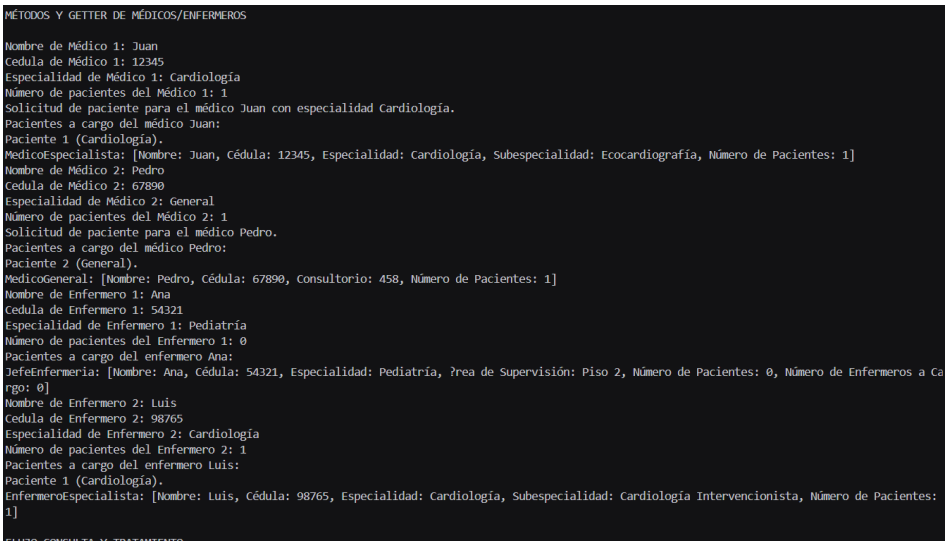


Figura 4: Representación de objetos sobrescribiendo el método `toString()`.

Se demuestra que las reglas de flujo clínico desarrolladas en la primera etapa (bloqueo de pacientes en consulta o tratamiento) se conservan intactas en las clases abstractas y son heredadas y respetadas por todas las nuevas subclases del personal médico.

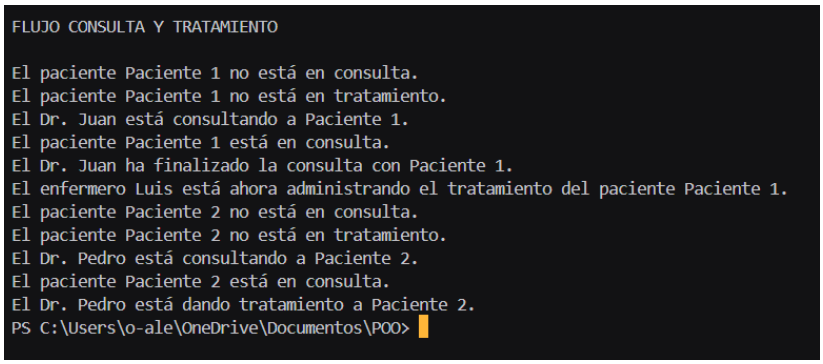


Figura 5: Conservación del flujo clínico heredado.

5. Conclusiones

La transición hacia el uso de interfaces y clases abstractas resolvió el problema de la falta de estandarización, la correcta aplicación del encapsulamiento, mediante la jerarquización de modificadores de acceso (`protected` y `private`), dio solución a la vulnerabilidad de los datos. Esto permitió proteger el estado interno de los objetos frente a manipulaciones externas no autorizadas desde el controlador central, delegando la inicialización segura a las superclases a través del método `super`. La herencia demostró su valor práctico al resolver la redundancia de código, facilitando la creación de roles especializados con comportamientos únicos (como la gestión de personal) sin duplicar la lógica base.

Finalmente, el resultado técnico más significativo fue la resolución del alto acoplamiento en la clase `Sistema` gracias al polimorfismo dinámico. En lugar de depender de estructuras de datos fragmentadas para cada tipo de médico, el polimorfismo permitió utilizar colecciones unificadas. La evaluación de tipos en tiempo de ejecución mediante `instanceof` y el despacho dinámico de métodos sobrescritos (`@Override`) proveen la solución directa para transformar un código rígido en una arquitectura robusta, cohesionada y lista para escalar.

Referencias

[1] Abstracción en POO. (s. f.). https://platzi.com.https://platzi.com/blog/abstraccion-en-poo-2/

[2] API de Java. (s.f.). IBM. Recuperado 8 de octubre de 2026, de https://www.ibm.com/docs/es/product-master/12.0.0?topic=code-java-api

[3] Gálvez, J. A. S. (s.f.). Organización de clases. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3088/mod_resource/content/1/UAPA-Organizacion-Clases/index.html

[4] Object (Java Platform SE 8). (2026, 21 julio). https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html

[5] Olguín, Abstracción y encapsulamiento. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3076/mod_resource/content/1/UAPA-Abstraccion-Encapsulamiento/index.html

[6] Olguín, J. A. S. G. R. A. S. (s.f.-b). Herencia. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3057/mod_resource/content/1/UAPA-Herencia/index.html

[7] Olguín, J. A. S. G. R. A. S. (s.f.-c). Polimorfismo. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3062/mod_resource/content/1/UAPA-Polimorfismo/index.html

[8] Programador Certificado Java 2. (2.a ed.). (1992). Alfaomega Grupo Editor.

[9] Sandoval, E. (2023, 21 septiembre). ¿Qué es un objeto en programación? Ebac. https://ebac.mx/blog/objeto-en-programacion